Devin SaJeun DuCharme

CS 790: Domain Specific Programming for AI Apps

Homework 3 Report

1) Conceptual Questions

Question 1: Trace-based JIT operates on the basis that a function operates on a specific path, and it is well equipped to handle functions that operate the same with every iteration, generally less complex models. Script-based JIT does a more overall analytical approach by interpreting the entire code to be able to preserve functionality of each loop or branch, establishing more reliability with functions that can change with altering inputs with complex dynamic behavior. This is especially noticeable with if statement loops such as:

```
if x > 7:
        return x * 5
else:
        return x + 2
```

Scripting would work because it can see and understand both branches of the if statement. Tracing can fail because if you utilized an input such as x = 11 first then x = 3 after, it may run x = 3 under x * 5 because that's what it understood to accomplish in the first iteration, not needing the x + 2.

Question 2: Functionally, JAX operates best when the computations are expressed as an array operation, rather than python-side loops and mutations that operate step-by-step. JAX is optimized to run large-scale mathematical operations on the GPU, which becomes inefficient performance-wise when the python operations occur outside of JAX. This can be fixed by eliminating the for loop and append operation to allow vectorized computation, so batching occurs at an array level rather than individualistic python loops.

Question 3:

(a) Eager mode enables each operation into its own GPU kernel, which allows step-by-step functionality. Therefore, with y = sin(cos(x)) + exp(x), there are four total GPU kernel launches:

1) a = cos(x)
2) b = exp(x)
3) c = sin(a)
4) y = c + b

Based on these restructured kernels, there are four writes (a,b,c,y) and five reads (x,x,a,c,b)

(b) Fused mode combines multiple operations into one GPU kernel, enhancing efficiency while minimizing global memory utilization. Therefore, the entire y = sin(cos(x)) + exp(x) is operated on in its entirety rather than piece-by-piece, establishing one kernel launch, one write (y), and one read (x).

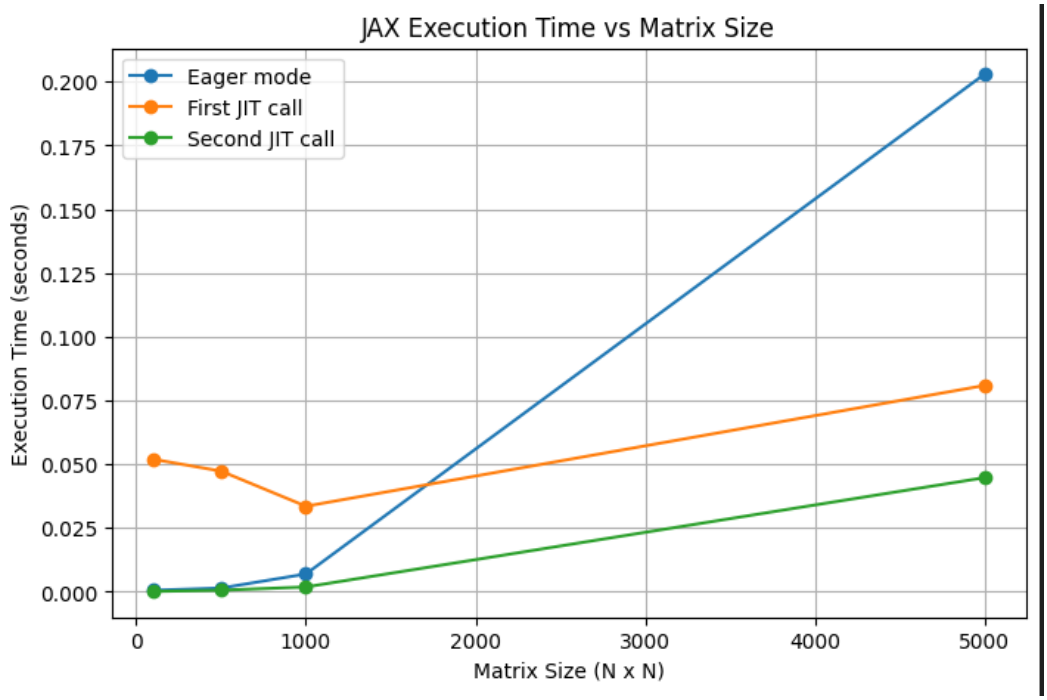
(c) The theoretical memory bandwidth reduction factor is the measure of how much less traffic occurs when going from eager mode to fused mode, essentially measuring efficiency. This is calculated by dividing memory access (read,write) from eager mode to fused mode. Given that there are 9 total reads/writes from eager mode and 2 total read/write from fused mode, global memory is reduced by a factor of 4.5 times.

Question 4: Within JIT compilation, dynamic control flow struggles due to the execution path relying on the tensor values which may alter operations at runtime. This goes against the compiler's preference to have fixed computations ahead of time, regressing optimization. Under JAX, python if statements do not perform as intended due to JAX's trace-based approach mentioned in question 1. Therefore, JAX has special control-flow primitives that allow the compiler to understand specific looping and branching that occur in if statements. PyTorch addresses this with script-based tools and therefore is more flexible, but may still run into issues if the program contains highly dynamic behavior.

Question 5: While running torch.compile for the first time, PyTorch compiles the function prior to execution which increases the runtime. During this first run, PyTorch traces the function, performs guard generation, optimizes the graph, generates kernels, and caches the compiled result. All following executions skip the compilation steps and strictly runs a guard check for verification and re-executes the cached compiled kernels, allowing for a expedited process that is optimized.

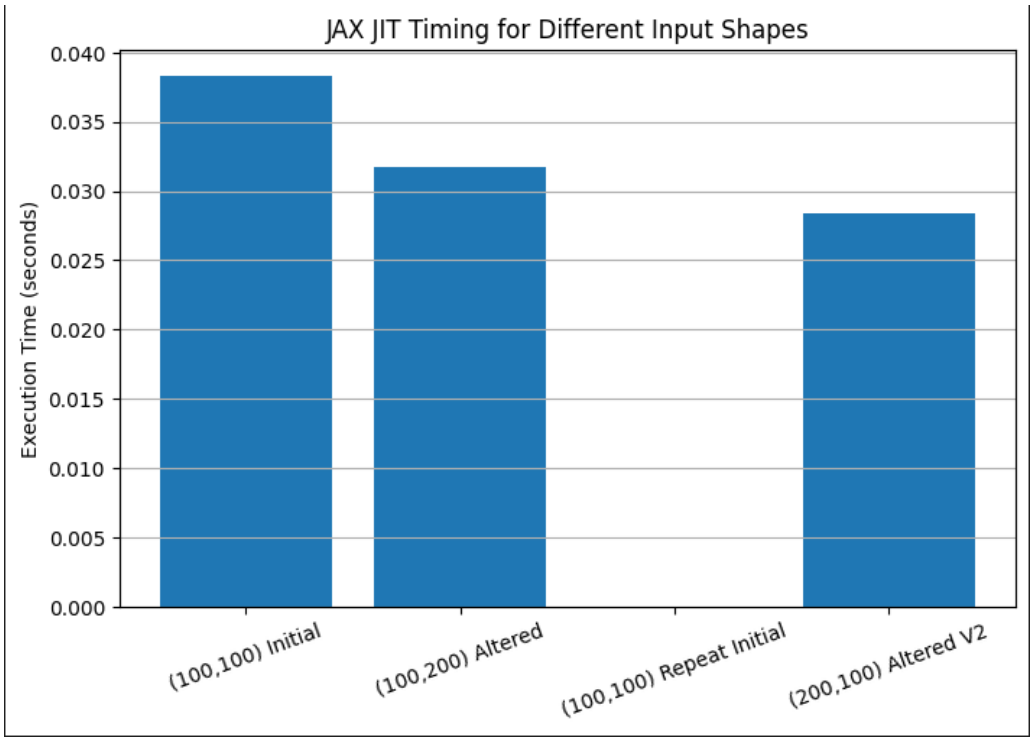
2) **Programming: JAX JIT Deep Dive**

Part 1: Measuring Compilation Overhead



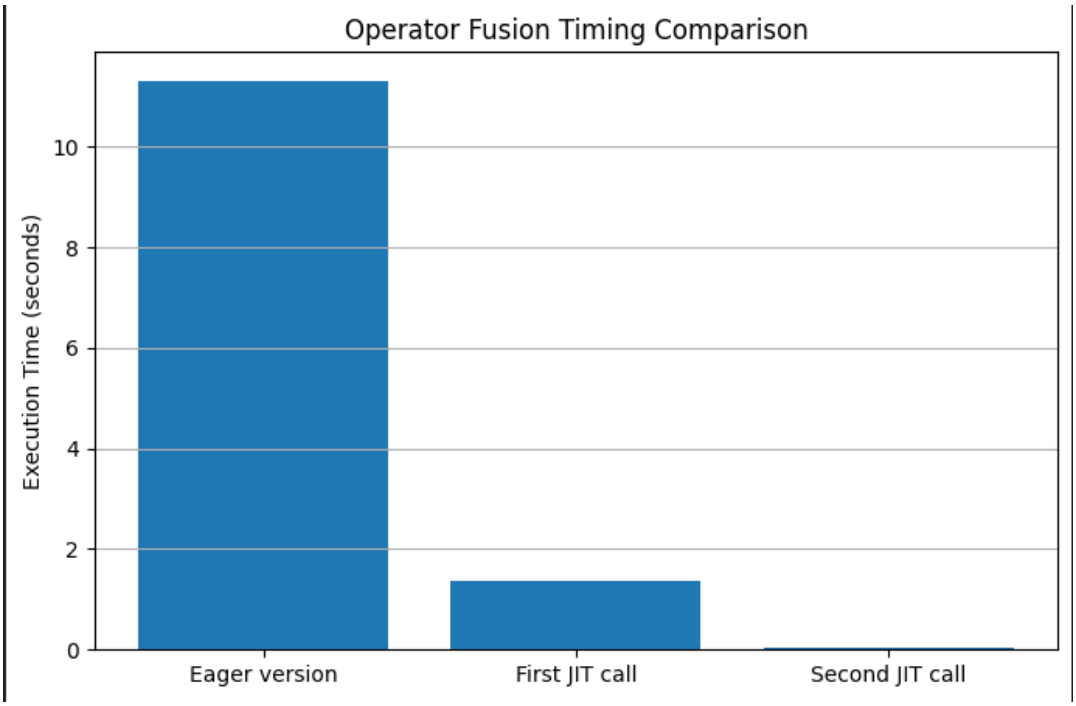
Analysis: The results demonstrate the trade-off between JIT compilation overhead and execution efficiency. Eager execution time increases with matrix size because operations are executed sequentially without optimization. The first JIT call is slower due to compilation overhead, which is relatively constant across input sizes since it depends on tracing and compiling the function rather than the amount of data. The second JIT call is the fastest because it reuses the compiled function, eliminating compilation cost. Its execution time increases with matrix size but remains significantly lower than eager mode due to optimized execution. Overall, JIT provides the greatest benefit for larger inputs, where computation dominates and compilation overhead becomes negligible.

Part 2: Shape Specialization



Analysis: JAX retraces and recompiles when the input shape changes. In this experiment, the first call with (100,100) is slow because it includes compilation. The calls with new shapes (100,200) and (200,100) are also slow because JAX must compile new versions for those shapes. The repeated call with (100,100) is much faster because JAX reuses the cached compiled function. The performance impact is that varying input shapes add compilation overhead, while repeated use of the same shape gives much faster execution. The JAXPR output confirms that JAX traces the function with shape-specific input signatures, which helps explain why different input shapes trigger retracing.

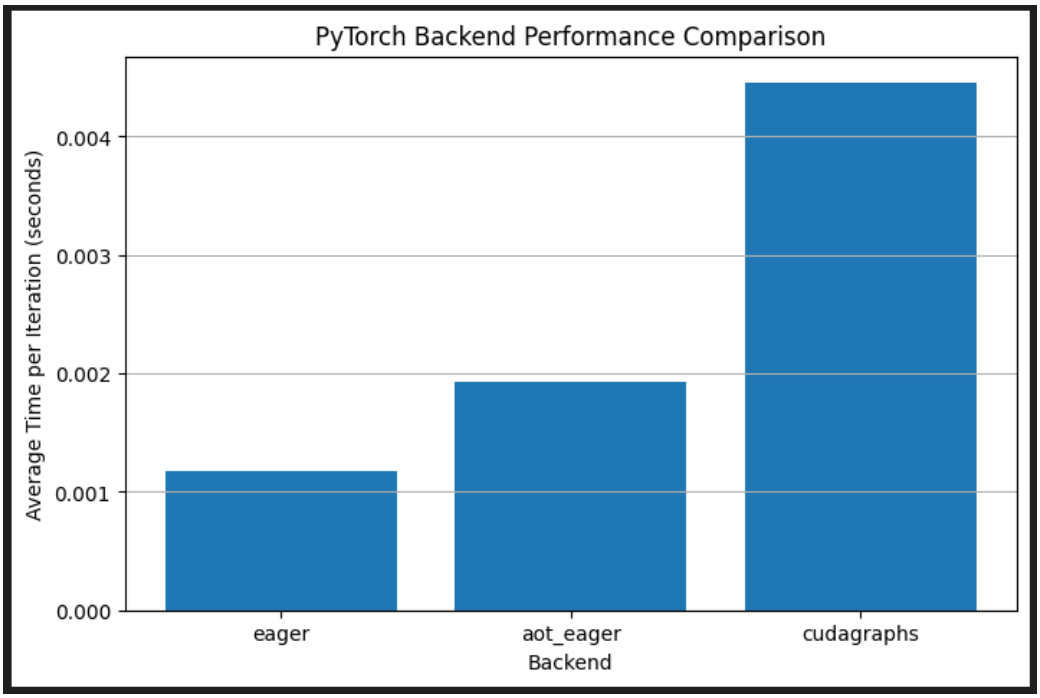
Part 3: Operator Fusion Analysis



Analysis: The JIT version achieves better performance because JAX traces the entire computation and compiles it into an optimized execution graph, allowing for operator fusion. In eager mode, the repeated elementwise operations are executed separately, leading to higher overhead and increased memory traffic. In contrast, the JIT-compiled version fuses many of these operations together, reducing intermediate memory access and improving efficiency. While the exact number of fused operations could not be directly measured due to lack of nvprof support in the current environment and the incompatibility of PyTorch's profiler with JAX, the significant reduction in execution time indicates that substantial fusion occurred strictly from definition and expectation. The theoretical speedup comes from reducing thousands of separate operations into a smaller number of optimized compiled computations. Based on experimental results, the actual speedup was approximately over 200 times faster (11.315s vs. 0.054s), demonstrating the effectiveness of JIT compilation and operator fusion. The first JIT call was slower due to compilation overhead, while subsequent calls benefited from cached execution.

3) **PyTorch compile Analysis**

Part 1: Comparing Backends



Analysis: Different backends have different performance because they apply different levels of graph capture and optimization, and those optimizations do not always reduce runtime for every workload. After some recalibration, operations moved from the CPU to the GPU to harness CUDA utilization. In this experiment, “eager” performed best, while “aot_eager” was slightly slower and “cudagraphs” was significantly slower. This seems to suggest that, for this particular network and workload size, the extra compilation or graph-management overhead was larger than the performance benefit. Since the model is relatively small and each iteration is already very fast, there is less opportunity for advanced backends to provide meaningful speedup. The tradeoff is that “eager” has no compilation overhead and can perform well on smaller or simpler workloads, while compiled backends may perform better on larger, more stable workloads where graph capture and optimization can amortize their extra setup cost. “cudagraphs” is designed to reduce GPU launch overhead, but in this case its overhead appears to outweigh its benefit. The “inductor” backend could not be evaluated because a working Triton installation was not available in the current Windows environment, as I believe it is only Linux accessible.

Part 2: Debugging Compilation Failures

Analysis:

Problem 1 fails to compile because it uses data-dependent control flow through the condition ”if x.sum() > 0”. This introduces a graph break, as the branch depends on the runtime value of a tensor. The output confirms this with a non-zero graph break count and an error indicating unsupported dynamic control flow. The fixed version replaces the Python conditional with “torch.where”, which expresses the logic as a tensor operation and allows successful compilation with no graph breaks.

Problem 2 compiles successfully in both the broken and fixed versions, with no graph breaks reported. However, the use of a Python dictionary introduces unnecessary Python-side logic around a simple tensor computation. While PyTorch is able to handle this case, the fixed version removes the dictionary and produces a cleaner, more compiler-friendly implementation.

Problem 3 also compiles successfully without graph breaks, but it is written using a Python loop with repeated tensor operations. This structure is less efficient and may limit optimization opportunities. The fixed version vectorizes the computation, allowing the operation to be expressed as a single tensor expression, which is more suitable for compiler optimization.

Part 3: Graph Capture and Inspection

Analysis:

Which operations appear in the graph?

The tensor operations: two matrix multiplications, ReLU, residual addition, and layer normalization.

Which operations are not captured? Why?

The Python print statement and Python list modification are not captured because they are Python-side actions, not tensor operations.

What happens to the print statement?

It causes torch.compile(..., fullgraph=True) to fail and is reported by torch._dynamo.explain() as a break reason.

What happens to the list modification?

It remains a Python-side effect and is not part of the tensor graph.

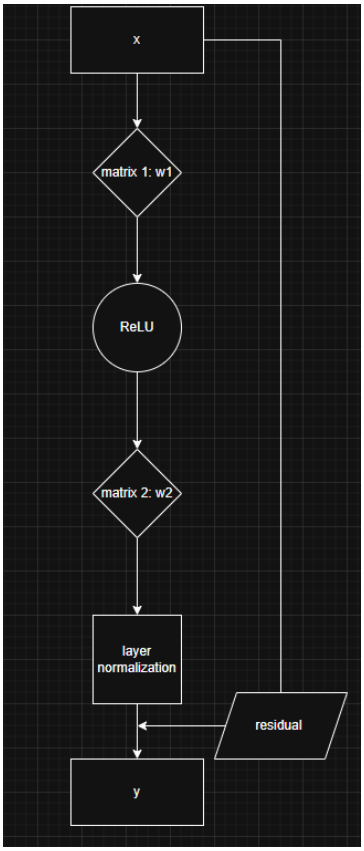



Diagram: